

CSC10014 · PRACTICAL TRACK · 2026

Lab 1

Developer Environment & Reproducible Setup

Terminal, VS Code, Python environments, repository structure, README and first commits

Practical Track · Week 1 · 2.5–3 h · Project: Smart Virtual Assistant

Faculty of Information Technology · University of Science, VNU-HCM

Instructor: Dr. LE Thanh Tung

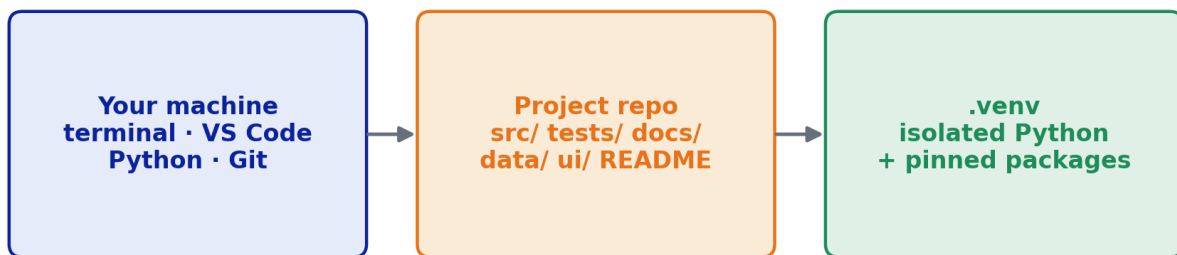
Contents

About This Lab

Lab 1 — Developer Environment, Command Line & Reproducible Setup. Practical track, Week 1. Duration: 2.5–3 hours in class, plus homework.

Goal

Set up a professional development environment on your own, move around a project confidently from the terminal, and make your work **reproducible**: a teammate must be able to run your project by following your README — without asking you anything.



Reproducible = a teammate gets the same result by following the README only.

Figure 0.1 — The three things you set up today.

Learning outcomes

1. Use the terminal for everyday tasks: navigate, create, copy, move, delete, search, pipe and redirect.
2. Understand paths, permissions and environment variables.
3. Create and use a Python virtual environment and install pinned dependencies.
4. Organise a repository with `src/`, `tests/`, `docs/`, `data/`, `ui/` and a correct `.gitignore`.
5. Write a README that works as an onboarding contract.
6. Make your first meaningful commits and push a tagged release.
7. Use an AI coding assistant responsibly: ask → inspect → verify → commit.

What you hand in

Kind	Deliverable	Where
Individual, in class	your fork of the starter repo runs; README setup section written; first commits; tag <code>v0.1</code>	GitHub link on the LMS
Individual, in class	pair verification sheet completed by your partner	<code>docs/lab1/pair_verification.md</code>
Team, in class	common project repository with the agreed folder skeleton	team repo, tag <code>v0.1-setup</code>
Individual, homework	"break your setup" log	<code>docs/lab1/setup_failure_log.md</code>
Individual, homework	Python bootcamp Week 1	see the Git & GitHub tutorial, Chapter 18

Materials

The instructor provides the **lab01-starter** repository and the worksheets in **lab01/worksheets/**:

```
lab01-starter/
├── README.md           # setup section is TODO – you write it
├── pyproject.toml
├── requirements.txt    # pinned dependencies
├── src/assistant/      # tiny rule-based assistant (no AI yet)
├── data/offices.csv
├── tests/test_smoke.py
├── docs/ ui/
└── scripts/check_env.py # tells you what is still wrong
```

IN OUR COURSE

Teams have **four members**. Individual work is done in your own fork or branch; team work goes into the common team repository.

Timeline for the session

Time	Activity
0:00–0:20	Workstation mindset; install check
0:20–0:55	Terminal essentials (Part B)
0:55–1:15	VS Code and the integrated terminal (Part C)
1:15–1:55	Lab steps 1–6: clone → environment → run → README → commit
1:55–2:25	Step 7: pair verification
2:25–2:50	Team: create the common project skeleton, tag v0.1-setup
2:50–3:00	Submit links; explain homework

Before Class

Do these at home. They take 30–60 minutes and save the whole lab.

1. Install **Python 3.10 or newer** (python.org on Windows/macOS; `sudo apt install python3 python3-venv` on Ubuntu).
2. Install **Git** and set `user.name` and `user.email` (Git tutorial, Chapter 2).
3. Create a **GitHub** account with your student email, turn on 2FA, and add an SSH key.
4. Install **VS Code** and the extensions *Python* and *Pylance* (Microsoft).
5. Read *The Missing Semester — The Shell* (45–60 min) and GitHub *Hello World* (30 min).

Check:

```
$ python --version          # Windows may use: py --version
Python 3.12.3
$ git --version
git version 2.47.1
$ ssh -T git@github.com
Hi minh-nv! You've successfully authenticated, but GitHub does not provide shell access.
```

TIP

On Windows use **Git Bash** for the commands in this tutorial — they are the same as on macOS and Linux. PowerShell equivalents are given where they differ.

Part A — The Developer Workstation Mindset

A.1 Local vs. remote environments

	Local	Remote
Where code runs	your laptop	a server, container or cloud machine
Examples	VS Code + terminal + <code>.venv</code>	GitHub Codespaces, a university VM, CI runners
Strength	fast, offline, full control	identical for everyone, powerful hardware
Risk	"works on my machine"	cost, network dependence

Professional teams make the local environment **reproducible**, so that local, CI and production behave the same way. Today you build that habit.

A.2 Three rules of a reproducible project

1. **Everything needed to run is written down** — in the README and in dependency files, not in someone's head.
2. **Dependencies are isolated and pinned** — each project has its own virtual environment and exact versions.
3. **Nothing personal is committed** — no `.venv`, no secrets, no machine-specific paths.

A.3 Check yourself

CHECK YOURSELF

- 1) Name one advantage of a remote environment.
- 2) Why is "it works on my laptop" not good enough for a team of four?
- 3) Which of these belongs in Git: `.venv/`, `requirements.txt`, `.env`, `README.md`?

Part B — Terminal Essentials

B.1 Where am I, what is here?

```
$ pwd
/home/minh/projects
$ ls
lab01-starter  notes.txt
$ ls -la          # long format, including hidden files
$ cd lab01-starter
$ cd ..          # up one level
$ cd ~           # home folder
$ cd -           # back to the previous folder
```

Bash / Git Bash / zsh	PowerShell	Meaning
<code>pwd</code>	<code>pwd / Get-Location</code>	print working directory
<code>ls, ls -la</code>	<code>ls, ls -Force</code>	list files
<code>cd path</code>	<code>cd path</code>	change directory
<code>mkdir -p a/b</code>	<code>mkdir a/b</code>	create folders
<code>cp a b, cp -r d1 d2</code>	<code>cp a b, cp -r d1 d2</code>	copy
<code>mv a b</code>	<code>mv a b</code>	move / rename
<code>rm file, rm -r dir</code>	<code>rm file, rm -r dir</code>	delete (no recycle bin!)
<code>cat file</code>	<code>cat file / Get-Content</code>	show a file
<code>grep word file</code>	<code>Select-String word file</code>	search text
<code>find . -name "*.py"</code>	<code>ls -r -filter *.py</code>	find files
<code>echo \$HOME</code>	<code>echo \$env:USERPROFILE</code>	show a variable

WARNING

`rm -r` deletes immediately and permanently. Never run `rm -rf` on a path you have not checked with `pwd` and `ls` first.

B.2 Create, copy, move

```
$ mkdir -p sandbox/data sandbox/docs
$ cd sandbox
$ echo "hello" > docs/note.txt
$ cp docs/note.txt docs/note_copy.txt
$ mv docs/note_copy.txt data/
$ ls -R
.:
data docs
./data:
note_copy.txt
./docs:
note.txt
```

B.3 Paths

- **Absolute** paths start from the root: `/home/minh/projects/lab01-starter` (Windows: `C:\Users\minh\projects\...`).
- **Relative** paths start from where you are: `data/offices.csv`, `../notes.txt`.
- `.` is the current folder, `..` the parent, `~` your home.
- Avoid spaces and Vietnamese diacritics in project paths — many tools break on them. Prefer `~/projects/csc10014/`.

B.4 Pipes and redirection

A pipe `|` sends the output of one command into the next. Redirection `>` writes output to a file (`>>` appends).

```
$ cat data/offices.csv | grep "Mon-Fri"
Training Office,I.101,Mon-Fri 07:30-16:30
Student Affairs,I.102,Mon-Fri 08:00-16:00
IT Helpdesk,E.005,Mon-Fri 08:00-17:00

$ grep -c "Mon-Fri" data/offices.csv
3
$ grep -rn "def " src/ > docs/functions.txt
$ find . -name "*.py" | wc -l
5
```

B.5 Searching code

```
$ grep -rn "reply" src/ # recursive, with line numbers
$ grep -rni "office" --include="*.py" .
$ find . -name "*.csv" -size -1M
```

B.6 Permissions (macOS / Linux)

```
$ ls -l scripts/check_env.py
-rw-r--r-- 1 minh minh 2143 Sep 26 10:12 scripts/check_env.py
$ chmod +x scripts/check_env.py
$ ./scripts/check_env.py
```

`r`, `w`, `x` mean read, write, execute for the owner, group and others. On Windows you usually run scripts with `python scripts/check_env.py` instead.

B.7 Environment variables

Environment variables are settings the operating system passes to programs — for example `PATH` (where to find commands) and, later, `LLM_API_KEY`.

```
$ echo $PATH
$ export APP_MODE=dev # PowerShell: $env:APP_MODE = "dev"
$ python -c "import os; print(os.environ.get('APP_MODE'))"
dev
```

NOTE

`export` lasts only for the current terminal. In Lab 3 you will use a `.env` file for project settings instead.

B.8 Terminal exercises

EXERCISE

B1 — In `sandbox/`, create `a/b/c`, put a file in `c`, move it to `a`, and delete `b` completely. Show the result with `ls -R`.

B2 — Count how many lines of `data/offices.csv` contain "Sat".

B3 — Save the list of all `.py` files in the starter repo into `docs/py_files.txt` with one command.

B4 — Set an environment variable `COURSE=CSC10014` and print it from Python.

Part C — VS Code as Your Workbench

C.1 Open the project the right way

Open the **folder**, not single files: *File* → *Open Folder...* → *lab01-starter*, or from the terminal:

```
$ cd lab01-starter
$ code .
```

C.2 Essential extensions

Extension	Why
Python (Microsoft)	run, debug, select interpreter
Pylance	type checking, auto-complete
Ruff	linting and formatting (Lab 3)
GitLens (optional)	history and blame in the editor

C.3 The integrated terminal and interpreter

1. Open the terminal with **Ctrl+`**. It starts in the project folder.
2. After creating **.venv** (Part D), press **Ctrl+Shift+P** → **Python: Select Interpreter** and choose **.venv**.
3. New terminals then activate the environment automatically.

C.4 Workspace settings for the team

Create **.vscode/settings.json** only if the team agrees to share it (otherwise keep **.vscode/** in **.gitignore**):

```
{
  "python.defaultInterpreterPath": ".venv/bin/python",
  "editor.formatOnSave": true,
  "files.trimTrailingWhitespace": true
}
```

TIP

Learn five shortcuts today: Ctrl+P (open file), Ctrl+Shift+F (search in project), F12 (go to definition), Ctrl+` (terminal), Ctrl+Shift+P (command palette).

Part D — Python Environments and Dependencies

D.1 Why a virtual environment?

Without isolation, every project shares one Python and one set of packages. Project A needs **pydantic 1**, project B needs **pydantic 2** — one of them breaks. A virtual environment is a private folder with its own Python and packages, one per project.

D.2 Create and activate

```
$ cd lab01-starter
$ python -m venv .venv
# macOS / Linux / Git Bash
$ source .venv/bin/activate
# Windows PowerShell
PS> .venv\Scripts\Activate.ps1
(.venv) $ python -c "import sys; print(sys.prefix)"
/home/minh/projects/lab01-starter/.venv
```

The **(.venv)** prefix shows the environment is active. **deactivate** leaves it.

WARNING

On Windows, PowerShell may block **Activate.ps1**. Run once: **Set-ExecutionPolicy -Scope CurrentUser RemoteSigned**, or use Git Bash.

D.3 Install pinned dependencies

```
(.venv) $ python -m pip install --upgrade pip
(.venv) $ pip install -r requirements.txt
(.venv) $ pip install -e .
(.venv) $ pip list
Package    Version
-----
assistant  0.1.0
pytest     8.3.3
...
```

- **requirements.txt** lists **exact** versions (**pytest==8.3.3**) so everyone gets the same result.
- **pip install -e .** installs the project itself in *editable* mode, so **python -m assistant** works from any folder and code changes apply immediately.

D.4 Adding a new dependency

```
(.venv) $ pip install rich==13.9.2
(.venv) $ echo "rich==13.9.2" >> requirements.txt
$ git add requirements.txt
$ git commit -m "chore(deps): add rich for coloured output"
```

TIP

`pip freeze > requirements.txt` captures everything installed, including sub-dependencies. It is exact but noisy; for this course, list your direct dependencies with pinned versions.

D.5 Optional: uv

uv is a fast, modern tool that creates environments and installs packages. It is optional in this course.

```
$ pip install uv
$ uv venv
$ uv pip install -r requirements.txt
```

D.6 Run the starter app and tests

```
(.venv) $ python -m assistant "where is the training office?"
Training Office: room I.101, open Mon-Fri 07:30-16:30.
(.venv) $ python -m assistant
Study assistant (starter). Type 'quit' to exit.
> hi
Hello! Ask me where an office is, or when it opens.
> quit
(.venv) $ pytest -q
.... [100%]
4 passed in 0.01s
```

D.7 Check your environment

```
(.venv) $ python scripts/check_env.py
[ OK ] Python >= 3.10 (found 3.12.3)
[ OK ] running inside a virtual environment
[ OK ] pytest installed
[ OK ] git available
[ OK ] inside a git repository
[FAIL] .gitignore present
      -> Lab 1 step 5
[FAIL] README has no TODO left
      -> Lab 1 step 5: write the setup instructions
[ OK ] git user.email set (minh.nv@student.hcmus.edu.vn)
[ OK ] starter app imports and answers

2 problem(s) found.
```

The two failures are expected — you fix them in the lab steps.

Part E — Repository Structure and .gitignore

E.1 The standard layout

Folder / file	Purpose	Committed?
<code>src/<package>/</code>	application code (backend)	yes
<code>tests/</code>	automated tests	yes
<code>docs/</code>	brief, blueprint, ADRs, logbook, reports	yes
<code>data/</code>	small, non-sensitive sample data	yes (small files only)
<code>ui/</code>	user interface (from Week 7)	yes
<code>scripts/</code>	helper scripts	yes
<code>README.md</code>	onboarding contract	yes
<code>requirements.txt</code> , <code>pyproject.toml</code>	dependencies, project metadata	yes
<code>.venv/</code>	local environment	never
<code>.env</code>	secrets and local settings	never

E.2 Write the .gitignore

```
# .gitignore
.venv/
__pycache__/
*.pyc
.pytest_cache/
.ruff_cache/
*.egg-info/
.env
.env.*
!.env.example
.DS_Store
.vscode/
data/raw/
```

Check that it works:

```
$ git status --ignored
Ignored files:
    .venv/
    src/assistant.egg-info/
```

Part F — The README as an Onboarding Contract

A README is a promise: *"Follow these steps and it will work."* If a teammate follows it and it fails, the README is broken — not the teammate.

F.1 What a good README contains

1. One paragraph: what the project is.
2. **Prerequisites** with versions (Python $\geq$ 3.10, Git).
3. **Setup** — exact commands, for macOS/Linux and Windows.
4. **Run** — one command and the expected output.
5. **Test** — one command and the expected result.
6. **Project structure** — one line per folder.
7. **Troubleshooting** — the problems you actually hit.

F.2 Example setup section

```
## Setup
Prerequisites: Python 3.10+, Git.

git clone git@github.com:<you>/lab01-starter.git
cd lab01-starter
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -e .

## Run
python -m assistant "where is the library?"
# -> Library: room B.201, open Mon-Sat 07:00-20:00.

## Test
pytest -q                        # -> 4 passed

## Troubleshooting
- "No module named assistant" -> you forgot `pip install -e .` or the venv is not active.
- PowerShell blocks Activate.ps1 -> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

TIP

Write the README **while** you set up, copying each command that worked. Writing it afterwards from memory always misses a step.

Part G — First Commits and a Release

If Git is new to you, read Chapters 3–4 and 8 of the Git & GitHub tutorial first.

```
$ git status
$ git add .gitignore
$ git commit -m "chore: add gitignore for python project"
$ git add README.md
$ git commit -m "docs: write setup, run and test instructions"
$ git push
$ git tag -a v0.1 -m "Lab 1: reproducible setup"
$ git push origin v0.1
```

IN OUR COURSE

A *meaningful* commit changes one thing and says why. "update" or "lab1" are not meaningful messages.

Part H — Using an AI Coding Assistant Responsibly

AI assistants are allowed in this course when used transparently. The loop is **ask** → **inspect** → **verify** → **commit**.

Step	What you do	Example
Ask	give context and constraints	"Write a pytest test for <code>reply()</code> when the office name is lower-case. Do not change <code>rules.py</code> ."
Inspect	read every line; is it what you asked?	the AI also changed <code>rules.py</code> — reject that part
Verify	run it; check edge cases	<code>pytest -q</code> ; try "TRAINING OFFICE?"
Commit	small commit; disclose	<code>test(rules): lower-case office lookup</code> + note in <code>docs/ai-use-log.md</code>

WARNING

Never paste secrets, tokens or classmates' personal data into an AI tool, and never commit code you cannot explain.

The Lab, Step by Step

Step 1 — Get the starter repository (individual)

1. Open the starter repository link from the LMS and click **Use this template** → **Create a new repository** (or **Fork**). Name it `lab01-<github-username>`.
2. Clone your copy:

```
$ mkdir -p ~/projects/csc10014 && cd ~/projects/csc10014
$ git clone git@github.com:<you>/lab01-<you>.git
$ cd lab01-<you>
```

Step 2 — Create the environment

```
$ python -m venv .venv
$ source .venv/bin/activate      # Windows: .venv\Scripts\Activate.ps1
```

Step 3 — Install dependencies

```
(.venv) $ pip install -r requirements.txt
(.venv) $ pip install -e .
```

Step 4 — Run the starter app and the tests

```
(.venv) $ python -m assistant "where is the IT helpdesk?"
(.venv) $ pytest -q
```

Write down every command that worked — you need them in Step 5.

Step 5 — Add .gitignore and write the README

1. Create `.gitignore` (Part E.2). Run `git status` — `.venv/` must not appear.
2. Replace every `TODO` in `README.md` with real instructions (Part F).
3. Run `python scripts/check_env.py` until every line is `[ OK ]`.

Step 6 — Make your first meaningful commits

At least three commits, each with a Conventional Commit message: `.gitignore`, `README`, and one small improvement of your choice (for example, a new office in `data/offices.csv` with a test). Push and tag `v0.1`.

EXERCISE

Improvement ideas: make `reply()` understand "help"; add two offices and a test for each; make the greeting list include "good morning". Keep it small.

Step 7 — Pair verification

1. Swap repository links with a partner from **another** team.
2. Your partner clones your repository into a new folder and follows **only your README**. You may not talk or point.
3. The partner fills in `lab01/worksheets/pair_verification.md` and sends it to you.
4. You fix the README, commit the fix, and add the completed sheet as `docs/lab1/pair_verification.md`.

TIP

Target: from clone to passing tests in **under 10 minutes**. If it takes longer, the README is missing something.

Step 8 — Team: create the common project skeleton

As a team of four, create the team repository for the Smart Virtual Assistant (Git tutorial, Section 8.2) with this skeleton. Each member adds one part through their own pull request.

```
assistant-teamNN/
├── README.md           # Member 1
├── .gitignore requirements.txt pyproject.toml # Member 1
├── src/assistant/      # backend – Member 2 (copy the starter package)
├── ui/README.md        # UI plan – Member 3
├── data/README.md      # data sources & owners – Member 3
├── tests/              # smoke tests – Member 4
├── docs/README.md docs/team.md # Member 4
└── scripts/check_env.py # Member 2
```

Member	Pull request
1	README, <code>.gitignore</code> , <code>requirements.txt</code> , <code>pyproject.toml</code>
2	<code>src/assistant/</code> and <code>scripts/check_env.py</code>
3	<code>ui/README.md</code> , <code>data/README.md</code> , sample <code>data/offices.csv</code>
4	<code>tests/</code> , <code>docs/README.md</code> , <code>docs/team.md</code> (names, member numbers, roles)

When all four PRs are merged, Member 1 tags `v0.1-setup`.

Homework

H1 — Break your setup, then recover (individual)

1. Break your setup on purpose: delete `.venv`, move the repository to another folder, uninstall `pytest`, or clone into a fresh folder.
2. Recover using **only** your README.
3. Record at least one failure and its fix in `docs/lab1/setup_failure_log.md` (template in `lab01/worksheets/`). If the README needed a change, commit it.

H2 — Python bootcamp, Week 1 (individual + team)

Follow Chapter 18 of the Git & GitHub tutorial: exercises W1-1 to W1-5 in

`python-bootcamp/members/<your_folder>/w1/`, one PR per member, reviewed by your rotation partner; team exercise T-W1.

H3 — Reading

MIT *Missing Semester: Shell Tools and Scripting*; GitHub *Hello World*. Write three lines in your reflection note: one command you will use daily, one you found surprising, one question.

How to Submit

What	When	How
Individual repo <code>lab01-<you></code> with tag <code>v0.1</code>	end of the session	repo link on the LMS
Pair verification sheet	end of the session	<code>docs/lab1/pair_verification.md</code> in your repo
Team skeleton, tag <code>v0.1-setup</code>	end of the session	Tech Lead submits the tag link
H1 setup failure log	23:59 the day before Lab 2	file in your repo; link on the LMS
H2 bootcamp W1	23:59 the day before Lab 2	merged PR in the team repo; tag <code>py-w1-done</code>

IN OUR COURSE

If it is not on GitHub by the deadline, it was not submitted. Unfinished in-class work: push what you have and open a **Draft PR** with a note.

Grading checklist

Criterion	Points
Starter runs; tests pass; <code>check_env.py</code> all OK	3
README lets a stranger set up in < 10 min (pair sheet)	3
<code>.gitignore</code> correct; no <code>.venv</code> or secrets committed	1
≥ 3 meaningful commits; tag <code>v0.1</code>	1
Team skeleton: 4 PRs, 4 authors, reviewed; tag <code>v0.1-setup</code>	1
Homework H1 log with a real failure and fix	1
Total	10

Troubleshooting

Problem	Fix
<code>python: command not found</code> (Windows)	use <code>py</code> , or reinstall Python with "Add to PATH" ticked
<code>python3</code> works but <code>python</code> does not (macOS/Linux)	use <code>python3 -m venv .venv</code> ; inside the venv <code>python</code> works
No module named <code>venv</code> (Ubuntu)	<code>sudo apt install python3-venv</code>
PowerShell: "running scripts is disabled"	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
No module named <code>assistant</code>	activate <code>.venv</code> ; run <code>pip install -e .</code>
<code>pip</code> installs into the wrong Python	always use <code>python -m pip ...</code> inside the venv
<code>.venv/</code> shows up in <code>git status</code>	add it to <code>.gitignore</code> ; if already committed: <code>git rm -r --cached .venv</code>
SSL or proxy errors on campus Wi-Fi	try another network; ask the TA for the proxy settings
VS Code uses the wrong interpreter	Ctrl+Shift+P → <i>Python: Select Interpreter</i> → <code>.venv</code>
Vietnamese characters garbled in CSV	open files with <code>encoding="utf-8"</code>

Self-Check

1. What does `pwd` print, and why is it useful before `rm -r`?
2. Explain the difference between `>` and `>>`.
3. Why do we pin dependency versions?
4. What does `pip install -e .` do?
5. Why must `.venv/` never be committed?
6. What makes a README an "onboarding contract"?
7. List the four steps of responsible AI use.

Answers. 1) The current folder — to be sure you delete in the right place. 2) `>` overwrites a file, `>>` appends. 3) So every teammate and CI get identical behaviour. 4) Installs the project in editable mode so imports work everywhere and edits apply immediately. 5) It is large, machine-specific and reproducible from `requirements.txt`. 6) Following it must produce a working setup without extra help. 7) Ask → inspect → verify → commit.

Resources

- MIT — *The Missing Semester of Your CS Education: The Shell; Shell Tools and Scripting* (missing.csail.mit.edu).
- GitHub Docs — *Hello World* (docs.github.com/en/get-started/start-your-journey/hello-world).
- Python docs — *venv — Creation of virtual environments; Installing packages* (packaging.python.org).
- VS Code docs — *Python in Visual Studio Code*.
- Course companion — *Git & GitHub for Team Projects*, Chapters 2–4 and 8.